

Konspekt z projektu numer 2 z przedmiotu

## **Metody sztucznej inteligencji 2**

Zastosowanie Monte Carlo Tree Search

do gry w warcaby  $8 \times 8$

z obowiązkowym biciem i damkami

bijącymi po całej przekątnej

Grzegorz Prasek, 327394

Mykhailo Shamrai, 329923

WARSZAWA 2026

# Spis treści

|   |                                       |   |
|---|---------------------------------------|---|
| 1 | Opis problemu . . . . .               | 1 |
| 2 | Wstępny przegląd literatury . . . . . | 1 |
| 3 | Hipotezy badawcze . . . . .           | 3 |
| 4 | Sposób weryfikacji hipotez . . . . .  | 4 |
| 5 | Harmonogram działań . . . . .         | 5 |
| 6 | Ogólny projekt techniczny . . . . .   | 6 |

## 1 Opis problemu

Przedmiotem projektu jest zastosowanie algorytmu **Monte Carlo Tree Search** (MCTS) w wariancie **Upper Confidence Bound applied to Trees** (UCT) do stworzenia sztucznej inteligencji grającej w warcaby na planszy  $8 \times 8$ . Warcaby są grą deterministyczną, dwuosobową, z pełną informacją.

Rozpatrywany wariant gry obejmuje następujące zasady:

- **Plansza  $8 \times 8$**  z pionkami ustawionymi na ciemnych polach trzech pierwszych rzędów każdego gracza (po 12 pionków na stronę).
- **Obowiązkowe bicie** - zarówno pojedyncze, jak i wielokrotne (sekwencyjne). Jeżeli gracz ma możliwość bicia, musi z niej skorzystać. W przypadku wielokrotnego bicia gracz musi kontynuować sekwencję bić tak długo, jak jest to możliwe.
- **Damki** - pionek, który dotrze do ostatniego rzędu przeciwnika, staje się damką. Damka bije po całej przekątnej, tzn. może przeskakiwać bity pionek z dowolnej odległości i lądować na dowolnym wolnym polu za nim. Damka musi również wykonywać wielokrotne bicie.
- **Warunek zakończenia gry** - gra kończy się wygraną jednego z graczy, gdy przeciwnik nie ma żadnych ruchów (utracił wszystkie pionki lub jest zablokowany). Aby uniknąć nadmiernie długich rozgrywek, wprowadzony zostanie limit maksymalnej liczby ruchów (np. 150), po przekroczeniu którego partia zostanie uznana za remis.

Warcaby charakteryzują się umiarkowanym współczynnikiem rozgałęzienia, wynoszącym przeciętnie około 8 w pozycjach bez bicia, lecz znacznie mniejszym w pozycjach z obowiązkowym biciem, gdzie liczba legalnych ruchów może spaść nawet do 1. Celem projektu jest zbadanie skuteczności podstawowego algorytmu UCT oraz dwóch jego modyfikacji opisanych w artykule przeglądowym [5], a także porównanie ich z podejściem heurystycznym opartym na algorytmie alfa-beta.

## 2 Wstępny przegląd literatury

**MCTS i UCT.** Kompleksowy przegląd algorytmu MCTS oraz jego modyfikacji i zastosowań został opracowany przez Świechowskiego i in. [5]. Algorytm MCTS, zaproponowany w 2006 roku, łączy przeszukiwanie drzewa gry z losowymi symulacjami (*playouts*), budując drzewo w sposób asymetryczny - koncentrując się na najbardziej obiecujących gałęziach. Każda

iteracja MCTS składa się z czterech faz: (1) selekcji - algorytm schodzi po drzewie, wybierając w każdym węźle akcję o najwyższej wartości UCT; (2) ekspansji - do drzewa dodawany jest nowy węzeł; (3) symulacji - z nowego węzła wykonywana jest losowa rozgrywka do końca gry; (4) propagacji wstecznej - wynik symulacji jest propagowany w górę drzewa, aktualizując statystyki odwiedzonych węzłów. Formuła UCT w fazie selekcji ma postać:

$$a^* = \arg \max_{a \in A(s)} \left\{ Q(s, a) + C \sqrt{\frac{\ln N(s)}{N(s, a)}} \right\}, \quad (1)$$

gdzie  $Q(s, a)$  oznacza średni wynik akcji  $a$  w stanie  $s$ ,  $N(s)$  - liczbę odwiedzin stanu  $s$ ,  $N(s, a)$  - liczbę wyborów akcji  $a$  w stanie  $s$ , a  $C$  - stałą eksploracji kontrolującą równowagę między eksploatacją (pierwszy składnik) a eksploracją (drugi składnik).

**Modyfikacja 1: Hybryda MCTS–minimax.** Baier i Winands [1] zaproponowali modyfikację fazy symulacji algorytmu MCTS przeznaczoną dla gier z pułapkami taktycznymi. W standardowym UCT faza symulacji polega na wykonywaniu losowych ruchów aż do osiągnięcia stanu terminalnego. Problem polega na tym, że losowe symulacje nie uwzględniają odpowiedzi przeciwnika - w grze takiej jak warcaby, gdzie obowiązkowe bicie może prowadzić do wymuszonej sekwencji wymian, losowy playout generuje wyniki silnie zaszumione i słabo skorelowane z faktyczną jakością pozycji.

Metoda hybrydowa zastępuje losową symulację **przeszukiwaniem minimax** wykonywanym z węzła liścia drzewa MCTS. Przeszukiwanie minimax rozpatruje ruchy obu graczy naprzemiennie do ustalonej głębokości  $d$ , przyjmując, że gracz maksymalizujący wybiera ruch o najwyższej wartości, a przeciwnik - o najniższej. Kluczową cechą tego podejścia jest to, że minimax nie wymaga żadnej funkcji ewaluacyjnej - jeżeli przeszukiwanie dotrze do stanu terminalnego (wygrana, przegrana lub remis), zwraca wartość tego stanu; w przeciwnym razie wynik jest propagowany na podstawie wartości stanów terminalnych znalezionych w poddrzewie. Dzięki temu metoda zachowuje jedną z głównych zalet MCTS - brak konieczności posiadania wiedzy dziedzinowej - jednocześnie znacząco poprawiając jakość oceny pozycji w porównaniu z losowymi symulacjami.

**Modyfikacja 2: Wcześniejsze zakończenie symulacji (*early termination*).** Lorentz [3] wykazał, że w wielu grach planszowych zastąpienie pełnych losowych symulacji przez symulacje skrócone, zakończone heurystyczną oceną pozycji, poprawia skuteczność algorytmu MCTS. Intuicja jest następująca: w grze, która może trwać ponad 100 ruchów, losowa symulacja prowadzona do końca generuje wynik obarczony ogromnym szumem - po kilkudziesięciu losowych ruchach pozycja jest tak zdeformowana, że wynik końcowy niewiele mówi o jakości wyjściowego ruchu. Krótka symulacja (np. 10–20 ruchów) zakończona rozsądną

oceną heurystyczną daje bardziej informatywny sygnał.

Lanctot i in. [2] zaproponowali formułę łączącą klasyczną średnią z symulacji z oceną heurystyczną:

$$\hat{Q}(s, a) = (1 - \alpha) \cdot \frac{r_{s,a}}{n_{s,a}} + \alpha \cdot v_{s,a}, \quad (2)$$

gdzie  $r_{s,a}/n_{s,a}$  jest klasyczną średnią nagród z symulacji,  $v_{s,a}$  jest oceną heurystyczną pozycji obliczoną na podstawie ewaluacji minimax propagowanej niejawnie w drzewie, a  $\alpha \in [0, 1]$  jest parametrem wagowym. Przy  $\alpha = 0$  formuła redukuje się do standardowego UCT, a przy  $\alpha = 1$  - do czystej ewaluacji heurystycznej.

W kontekście niniejszego projektu jako funkcję heurystyczną zastosowaną przy odcięciu symulacji proponujemy prostą różnicę materiałową:

$$h(s) = (\text{pionki}_{\text{gracz}} + 3 \cdot \text{damki}_{\text{gracz}}) - (\text{pionki}_{\text{przec.}} + 3 \cdot \text{damki}_{\text{przec.}}), \quad (3)$$

znormalizowaną do przedziału  $[-1, 1]$ . Waga damki wynosi 3, ponieważ w rozpatrywanym wariancie gry damka bije po całej przekątnej i jest znacząco silniejsza od zwykłego pionka.

**Podejście heurystyczne - Chinook.** Klasycznym podejściem do gry w warcaby jest algorytm alfa-beta z ręcznie zaprojektowaną funkcją ewaluacyjną. Program Chinook [4] pokonał mistrza świata w warcabach, wykorzystując przeszukiwanie alfa-beta z funkcją uwzględniającą m.in. przewagę materiałową, pozycję damek, kontrolę centrum i mobilność. W niniejszym projekcie gracz heurystyczny oparty na alfa-beta z uproszczoną funkcją ewaluacyjną (inspirowaną podejściem Chinooka) posłuży jako punkt odniesienia. Funkcja będzie uwzględniać:

- **Przewagę materiałową** - pionek = 1 pkt, damka = 3 pkt.
- **Kontrolę centrum** - bonus za pionki na centralnych polach planszy.
- **Mobilność** - różnicę liczby legalnych ruchów obu graczy.
- **Bliskość promocji** - bonus za pionki bliskie linii awansu na damkę.

### 3 Hipotezy badawcze

W ramach projektu postawione zostały następujące hipotezy badawcze:

- **Hipoteza 1** (UCT vs. heurystyka): Gracz oparty na podstawowym algorytmie UCT (bez dodatkowych modyfikacji) osiąga porównywalną lub wyższą skuteczność niż gracz heurystyczny oparty na algorytmie alfa-beta, przy odpowiednio dużym budżecie obliczeniowym (liczbie iteracji MCTS).

- **Hipoteza 2** (hybryda MCTS–minimax): Hybryda MCTS–minimax, w której losowe symulacje zostają zastąpione płytkim przeszukiwaniem minimax na liściach drzewa, istotnie poprawia skuteczność gry w porównaniu z podstawowym UCT.
- **Hipoteza 3** (*early termination* z heurystyką): Wcześniejsze zakończenie symulacji z oceną heurystyczną (różnica materiałowa z uwzględnieniem wartości damek) poprawia jakość gry w porównaniu z pełnymi losowymi symulacjami, przy tym samym budżecie obliczeniowym.
- **Hipoteza 4** (wpływ parametru  $C$ ): Wartość stałej eksploracji  $C$  w formule UCT istotnie wpływa na skuteczność gry.

## 4 Sposób weryfikacji hipotez

### Gracze

W eksperymentach wezmą udział czterej gracze komputerowi:

1. **UCT podstawowy** - standardowa implementacja algorytmu UCT z losowymi symulacjami do końca gry (lub do limitu ruchów).
2. **UCT + minimax (hybryda)** - modyfikacja zaczerpnięta z pracy Baiera i Winandsa [1]. Faza symulacji zostaje zastąpiona płytkim przeszukiwaniem minimax o konfigurowalnej głębokości  $d$ .
3. **UCT + *early termination*** - modyfikacja inspirowana pracami Lorentza [3] i Lancotta i in. [2]. Symulacje zostają skrócone; po ustalonej liczbie ruchów pozycja jest oceniana funkcją heurystyczną (3).
4. **Gracz heurystyczny (alfa-beta)** - algorytm alfa-beta z iteracyjnym pogłębianiem i funkcją ewaluacyjną inspirowaną programem Chinook [4], uwzględniającą przewagę materiałową, kontrolę centrum, mobilność oraz bliskość promocji.

### Eksperymenty

**Turnieje między graczami.** Każda para graczy rozegra serię partii z naprzemiennym przydziałem kolorów. Ze względu na stochastyczny charakter algorytmu MCTS każda partia będzie przeprowadzona z innym ziarnem generatora liczb pseudolosowych. Program zostanie zaprojektowany tak, aby ziarno generatora było parametrem wejściowym, co umożliwi dokładne odtworzenie dowolnego przebiegu.

**Badanie wpływu parametru  $C$ .** Dla wybranego wariantu UCT przeprowadzona zostanie seria eksperymentów z wartościami stałej eksploracji  $C$  z zakresu od 0.1 do 3.0. Dla każdej wartości  $C$  rozegranych zostanie co najmniej 50 partii.

**Badanie wpływu budżetu obliczeniowego.** Skuteczność każdego wariantu UCT zostanie zbadana dla różnych liczb iteracji (np. 100, 500, 1000, 5000, 10000), aby ocenić, jak jakość gry skaluje się z budżetem obliczeniowym.

**Testy z ludźmi.** Dostarczony zostanie interfejs graficzny umożliwiający rozgrywkę człowiek–komputer. Rozgrywki testowe zostaną przeprowadzone z udziałem co najmniej 5 różnych osób. Każda osoba rozegra co najmniej 2 partie (jedną jako biały, jedną jako czarny). Zebrane zostaną informacje o wynikach partii oraz subiektywnej ocenie siły gry komputera.

## Miary jakości

- Procent wygranych (*win rate*) w serii partii.
- Średnia długość partii (w liczbie ruchów).
- Średni czas obliczeń na ruch.

## Powtarzalność eksperymentów

Ze względu na stochastyczny charakter MCTS, każdy eksperyment zostanie powtórzony wielokrotnie z różnymi ziarnami generatora liczb pseudolosowych. Wyniki zostaną przedstawione z podaniem średniej, odchylenia standardowego oraz przedziałów ufności. Program będzie zaprojektowany tak, aby każdy przebieg mógł zostać dokładnie odtworzony przez podanie tego samego ziarna.

## 5 Harmonogram działań

- **Tydzień 1–2:** Implementacja silnika gry w warcaby (reprezentacja planszy, generowanie legalnych ruchów z uwzględnieniem obowiązkowego bicia i damek, wykrywanie końca gry). Implementacja podstawowego algorytmu UCT. Testy poprawności. Implementacja modyfikacji: hybryda MCTS–minimax oraz UCT z *early termination*. Implementacja gracza heurystycznego (alfa-beta). Weryfikacja poprawności.
- **Tydzień 3:** Implementacja interfejsu graficznego (Pygame) do gry człowiek–komputer. Przygotowanie infrastruktury eksperymentalnej (skrypty do automatycznych turniejów, zbieranie wyników do plików CSV).

- **Tydzień 4:** Przeprowadzenie eksperymentów: turnieje między graczami, badanie wpływu parametrów. Testy z ludźmi, analiza wyników, przygotowanie wizualizacji i tabel porównawczych. Napisanie raportu końcowego.

## 6 Ogólny projekt techniczny

Program zostanie zaimplementowany w języku Python. Planowane jest wykorzystanie następujących bibliotek:

- **NumPy** - do wydajnych operacji macierzowych.
- **Pygame** - do stworzenia interfejsu graficznego umożliwiającego grę człowiek-komputer.
- **Matplotlib** - do wizualizacji wyników eksperymentów.
- **Pandas** - do analizy i agregacji wyników zapisywanych w plikach CSV.

Architektura programu zakłada modularną budowę: oddzielny moduł silnika gry (reprezentacja stanu, generowanie ruchów), oddzielne moduły graczy (UCT, alfa-beta), moduł interfejsu graficznego oraz skrypty eksperymentalne. Program będzie przyjmował jako parametry wejściowe: wariant gracza, budżet obliczeniowy (liczba iteracji), wartość stałej  $C$ , głębokość minimax/odcięcia oraz ziarno generatora liczb pseudolosowych.



# Bibliografia

- [1] Hendrik Baier i Mark H.M. Winands. „MCTS-Minimax Hybrids”. W: *IEEE Transactions on Computational Intelligence and AI in Games* 7.2 (2014), s. 167–179.
- [2] Marc Lanctot i in. „Monte Carlo Tree Search with Heuristic Evaluations using Implicit Minimax Backups”. W: *2014 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 2014, s. 1–8.
- [3] Richard Lorentz. „Using Evaluation Functions in Monte Carlo Tree Search”. W: *Theoretical Computer Science* 644 (2016), s. 106–113.
- [4] Jonathan Schaeffer. *One Jump Ahead: Challenging Human Supremacy in Checkers*. Springer-Verlag, 1997.
- [5] Maciej Świechowski i in. „Monte Carlo Tree Search: a review of recent modifications and applications”. W: *Artificial Intelligence Review* 56 (2023), s. 2497–2562.